

ANGELES UNIVERSITY FOUNDATION

COLLEGE OF COMPUTER STUDIES

ANGELES CITY



Procedural Game Sound Effects (SFX) Generator in x86 Assembly with GUI Frontend

A Final Project Presented to

The Faculty, College of Computer Studies

Angeles University Foundation

In Partial Fulfillment of the Requirements for the Course

CA010 – Computer Architecture and Organization

Proponents

Allein Edward Miranda

Jan Emil Stenland

Jose Gian M. Tubera

Ray A. Nicolas

Instructor

OCTOBER 2026

Project Overview

This project creates a fast, lightweight sound effects generator for games. It combines an x86-64 Assembly backend with a clear graphical user interface (GUI). The assembly core generates retro audio waveforms like square, sawtooth, triangle, and white noise, applies pitch sweeps and volume envelopes, and saves the audio directly to 16-bit WAV format in memory. Users can adjust sound parameters through the GUI in real time.

Chapter 1: Introduction

1.1 Statement of the Problem

Sound effects are vital for player feedback in video games. However, modern game sound software is often bloated, slow to load, and reliant on heavy third-party engines for simple tasks.

In computer science coursework, assembly language exercises are often limited to basic console programs like text parsers or calculators. These assignments fail to show how low-level code handles fast data processing and audio work.

This project addresses the challenge of building a lightweight procedural audio generator using assembly code, connected to a user-friendly interface without sacrificing performance.

1.2 Objectives of the Study

1.2.1 General Objective

To create and test a procedural sound effects generator using an optimized x86-64 Assembly backend for sound synthesis, paired with an easy-to-use desktop GUI frontend.

1.2.2 Specific Objectives

- **Core Waveform Generation:** Implement assembly routines to generate square, sawtooth, triangle, and white noise sound waves directly in memory buffers.
- **Frequency and Volume Control:** Write math functions in assembly to support pitch slides, vibrato, and volume attack-decay envelopes.
- **Audio Export:** Format audio samples into standard 16-bit WAV files in memory for fast playback and saving.
- **GUI Frontend Integration:** Connect a desktop interface to the assembly code library, enabling real-time audio controls.
- **Performance Profiling:** Evaluate audio generation speed and memory usage compared to high-level implementations.

1.3 Scope and Limitations

Scope

- **Waveform Types:** Procedural creation of square, sawtooth, triangle, and white noise waveforms.
- **Audio Output:** Single-channel, 16-bit audio sampled at standard 44.1 kHz.
- **Sound Controls:** Configurable base frequency, pitch slides, vibrato, and volume envelopes.
- **Architecture:** Core sound engine built in x86-64 Assembly (NASM) and compiled into a dynamic library.
- **Interface:** Desktop interface with parameter sliders, audio presets, visual waveform displays, and WAV export.

Limitations

- **Operating System Mode:** Runs in standard user mode and sends audio buffers to the operating system rather than directly controlling sound hardware.
- **Monophonic Sound:** Supports one sound effect at a time rather than multi-note music composition.
- **Basic Effects:** Focuses on core audio generation without complex post-processing effects like reverb or 3D audio positioning.

1.4 Significance of the Study

- **For Computer Science Students:** Provides a practical application of low-level system design by connecting memory manipulation and registers directly with interactive media output.
- **For Game Developers:** Offers a standalone, lightweight tool for quickly generating sound effects without needing heavy software installations.
- **For Software Engineers:** Demonstrates how performance-critical assembly code can be linked with modern graphical frontends.

Chapter 2: Background / Rationale of the Study

This chapter lays the technical groundwork for the Procedural Game Sound Effects (SFX) Generator. It reviews the basics of x86-64 Assembly, the instructions the sound engine depends on, real-world uses of Assembly, and existing tools that do similar work. It then compares three common assemblers, explains why Assembly was chosen over high-level languages, and closes with the program architecture and functional specifications that will guide development.

2.1 Assembly Language Overview

Assembly language is a low-level programming language where each instruction maps almost directly to one machine instruction. An assembler translates readable mnemonics such as MOV, ADD, and JMP into binary machine code. Unlike high-level languages, Assembly gives the programmer direct control over registers, memory addresses, and the order of execution.

This project targets the x86-64 architecture (also called AMD64 or Intel 64), the 64-bit extension of the x86 family used by most desktop and laptop processors today.

2.1.1 Registers

Registers are small, very fast storage locations inside the CPU. Keeping working values in registers instead of memory is the main way Assembly code gains speed.

- **General-Purpose Registers:** Sixteen 64-bit registers (RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP, and R8 to R15). Their lower portions can be used as 32-, 16-, or 8-bit registers (for example, EAX, AX, and AL). In this project, they hold loop counters, buffer pointers, and integer sample values.
- **Instruction Pointer (RIP):** Holds the address of the next instruction. x86-64 supports RIP-relative addressing, which the engine uses to read constants and lookup tables.
- **Flags Register (RFLAGS):** Stores status bits (zero, carry, sign, and overflow) set by arithmetic and comparison instructions. Conditional jumps read these bits to make decisions.

- **SSE Registers (XMM0 to XMM15):** 128-bit registers used for floating-point and SIMD (Single Instruction, Multiple Data) operations. The engine uses them for phase, frequency, and volume calculations in single-precision floating point.
- **Stack Pointer (RSP) and Base Pointer (RBP):** Manage the stack, which stores return addresses, saved registers, and local variables during function calls.

2.1.2 Memory

- **Flat Memory Model:** In 64-bit user mode, a program sees one continuous virtual address space. The operating system maps this space to physical memory.
- **Program Sections:** Code is placed in the `.text` section, and read-only data such as lookup tables and the WAV header template in `.rodata`. The engine has no writable `.data` or `.bss` section: all working buffers are supplied by the caller, and all other state lives in registers or on the stack. This keeps the engine reentrant and its output deterministic.
- **Addressing Modes:** x86-64 supports the form `[base + index * scale + displacement]`. For example, `mov ax, [rdi + rcx * 2]` reads the 16-bit sample at position RCX in a buffer starting at RDI. This fits audio work well, since a sound is simply an array of fixed-size samples.
- **Little-Endian Byte Order:** x86 stores multi-byte values with the lowest byte first. The WAV format uses the same order, so header fields can be written directly without byte swapping.
- **Alignment:** Some SSE instructions (such as MOVAPS) require 16-byte aligned addresses, and both major calling conventions require the stack to be 16-byte aligned at each function call.

2.1.3 Control Flow

- **Comparisons and Conditional Jumps:** CMP and TEST set the flags, then instructions such as JE, JNE, JL, and JGE jump only if a condition is met. This is how Assembly expresses if-else logic.
- **Loops:** Loops are built from a counter and a conditional jump (for example, `dec rcx` followed by `jnz loop_start`). The per-sample loop that fills the audio buffer is the core of the engine.

- **Procedures:** CALL pushes the return address onto the stack and jumps to a routine; RET pops the address and returns.
- **Calling Conventions:** These rules define how arguments are passed and which registers a function must preserve. The Windows x64 convention passes the first four arguments in RCX, RDX, R8, and R9 (or XMM0 to XMM3 for floats) and requires 32 bytes of shadow space. The System V convention used by Linux and macOS passes them in RDI, RSI, RDX, RCX, R8, and R9 (or XMM0 to XMM7). The engine must follow the convention of its target platform so the GUI can call it like any normal library function.
- **Volatile and Nonvolatile Registers:** Under the Windows x64 convention, RAX, RCX, RDX, R8 to R11, and XMM0 to XMM5 are volatile: a function may change them freely. RBX, RBP, RDI, RSI, R12 to R15, and XMM6 to XMM15 are nonvolatile: a function that uses them must restore them before returning on every exit path. Breaking this rule corrupts the caller's state and typically crashes the GUI far from the actual bug. The engine is therefore designed to fit in the volatile registers, so that no saves are needed.

2.2 Instruction Set Breakdown

The sound engine relies on a small subset of the x86-64 instruction set. The table below groups these instructions by category and states how each group is used in the project.

Category	**Key Instructions**	**Use in the SFX Engine**
Data Movement	MOV, MOVZX, MOVSX, LEA, MOVSS, MOVAPS	Load sound parameters, compute buffer addresses, and move samples between registers and memory
Integer Arithmetic	ADD, SUB, INC, DEC, IMUL	Update loop counters, compute buffer offsets, and calculate WAV header sizes
Logic and Bit Shifts	AND, OR, XOR, SHL, SHR	Run the xorshift random number generator for white noise and mask indexes into the sine lookup table
Floating-Point Arithmetic (SSE)	ADDSS, SUBSS, MULSS, DIVSS, MINSS, MAXSS, ANDPS	Advance the waveform phase, apply pitch slides and vibrato, cap the frequency (MINSS), scale by the volume envelope, and take absolute values for the triangle wave

Category	Key Instructions	Use in the SFX Engine
Conversion	CVTSI2SS, CVTSS2SI, CVTTSS2SI	Convert sample counters to floating point and convert finished samples back to integers
Saturation	PACKSSDW	Clamp 32-bit results into the signed 16-bit range (-32,768 to 32,767) to prevent wrap-around distortion
Comparison and Branching	CMP, TEST, COMISS, JE, JNE, JL, JGE, JB, JAE, JP, JMP	Validate parameters (JP catches NaN values after COMISS), select the waveform, switch envelope stages, wrap the phase, and exit the sample loop
Stack and Procedures	PUSH, POP, CALL, RET	Save any nonvolatile registers that are used and call internal routines
Block Operations	REP STOSB, REP MOVSB	Clear output buffers and copy the WAV header template

The engine uses SSE for floating-point math instead of the older x87 FPU. Both x86-64 calling conventions pass floating-point arguments in XMM registers, and SSE avoids the x87 register stack, which makes the code simpler to write and debug. One trade-off is that SSE has no exponent instruction (x87 has F2XM1), so the pitch-slide multiplier is computed with a short polynomial instead, as described in Section 2.7.2.

2.3 Real-World Applications of Assembly Language

Assembly is rarely used to write entire applications today, but it remains essential wherever performance or direct hardware control matters.

- **Operating Systems and Bootloaders:** Kernels such as Linux use Assembly for boot code, interrupt handling, and switching between tasks, since these steps happen before or below what a high-level language can control.
- **Multimedia and Audio/Video Codecs:** Projects such as FFmpeg and the dav1d video decoder contain large amounts of hand-written SIMD Assembly for their most time-critical loops. This is the category closest to this project, since audio synthesis is also a tight loop over large arrays of samples.

- **Cryptography:** Libraries such as OpenSSL include Assembly versions of encryption and hashing routines to use special CPU instructions and to keep execution time constant, which protects against timing attacks.
- **Embedded Systems and Firmware:** Microcontrollers with very limited memory and clock speed often run Assembly routines for startup code and time-sensitive device control.
- **Game Development:** Classic consoles such as the NES and Game Boy were programmed largely in Assembly, including their sound drivers. On the PC, RollerCoaster Tycoon (1999) was written almost entirely in x86 Assembly by Chris Sawyer.
- **Performance Analysis and Security:** Developers read compiler-generated Assembly to find bottlenecks, and security researchers read it to reverse engineer software and analyze malware.

The common thread is that Assembly is used for small, critical parts of a system rather than for everything. This project follows the same pattern by writing only the sound engine in Assembly.

2.4 Related Systems and Tools

Existing tools fall into two groups: retro SFX generators written in high-level languages, and compact music synthesizers written in Assembly. No widely known tool combines both, which is the gap this project fills.

2.4.1 sfxr and Its Derivatives

- **sfxr:** Created by Tomas Pettersson (DrPetter) in 2007 for the Ludum Dare game jam. Written in C++, it generates retro sound effects from presets such as Pickup/Coin, Laser/Shoot, Explosion, Powerup, Hit/Hurt, Jump, and Blip/Select. It offers square, sawtooth, sine, and noise waveforms, an attack-sustain-decay envelope, frequency slides, vibrato, filters, and WAV export. It is the main inspiration for this project's parameter set and presets.
- **bfxr:** An expanded version by Stephen Lavelle that adds more waveforms and features, originally built in Flash (ActionScript).
- **jsfxr:** A JavaScript port of sfxr that runs in a web browser.

2.4.2 Assembly-Based Synthesizers

- **4klang:** A software synthesizer by the demoscene group Alcatraz, built for 4 KB intros (programs whose entire file must fit in 4,096 bytes). Its sound engine is written in x86 Assembly so that full songs can be played from only a few kilobytes of code.
- **Clinkster:** A compact synthesizer by Blueberry, also written in Assembly for size-limited demoscene productions.
- **Sointu:** A modern, cross-platform successor to 4klang that can compile its synthesizer to native x86 Assembly or WebAssembly.

2.4.3 Summary of Related Tools

Tool	Language	Purpose	Interface	Relation to This Project
sfxr	C++	Retro sound effects	Desktop GUI	Main model for parameters and presets
bfxr	ActionScript	Retro sound effects	Flash GUI	Shows demand for more waveforms and controls
jsfxr	JavaScript	Retro sound effects	Web browser	Shows the same engine can be ported easily
4klang	x86 Assembly	Music for size-limited intros	Plugin for music software	Proves Assembly synthesis is practical
Clinkster	x86 Assembly	Music for size-limited intros	Plugin for music software	Same as above, with a simpler design
This project	x86-64 Assembly (NASM) and GUI	Retro sound effects	Desktop GUI	Combines an Assembly engine with an sfxr-style tool

The sfxr family proves that a small set of parameters is enough to produce useful game sounds, but none of these tools use Assembly. The demoscene synthesizers prove that

Assembly can generate audio efficiently, but they are built for music in tiny programs, not as standalone sound effect tools. This project sits between the two.

2.5 Comparison of Assemblers: NASM, MASM, and TASM

NASM was selected as the assembler for this project. It is the only one of the three that is free, open source, cross-platform, and fully supports x86-64.

Feature	**NASM**	**MASM**	**TASM**
Full Name	Netwide Assembler	Microsoft Macro Assembler	Turbo Assembler
Developer	Open-source community (originally Simon Tatham and Julian Hall)	Microsoft	Borland
First Released	1996	1981	1989
License	Free and open source (BSD)	Proprietary, free with Visual Studio	Proprietary
x86-64 Support	Yes	Yes (through ML64)	No (16- and 32-bit only)
Platforms	Windows, Linux, macOS	Windows only	DOS and older Windows
Output Formats	Win64 COFF, ELF64, Mach-O, flat binary, and others	Windows COFF	DOS and Windows object files
Syntax Style	Intel syntax, strict and explicit	Intel syntax with high-level directives (mostly unavailable in ML64)	Intel syntax, with MASM-compatible and IDEAL modes
Macro Support	Strong preprocessor (%macro, %define)	Strong	Strong
Current Status	Actively maintained	Maintained with Visual Studio	No longer developed

Reasons for choosing NASM:

- **64-bit Requirement:** The engine targets x86-64, which rules out TASM.
- **Cross-Platform Output:** The same source can produce a Windows DLL or a Linux shared library, while MASM is limited to Windows.
- **Explicit Syntax:** NASM requires operand sizes to be stated (for example, `dword [rdi]`) and does not hide work behind high-level directives. This makes the code easier to read and better suited to a course on computer architecture.
- **Cost and Tooling:** NASM is free and works with common linkers such as GCC, MinGW, and the Microsoft linker.

2.6 Rationale for Choosing Assembly over High-Level Languages

The sound engine is written in Assembly because audio synthesis is a tight, repetitive loop where direct control over registers and memory has a visible effect. A one-second sound at 44.1 kHz requires 44,100 iterations of the same calculation, so every instruction inside that loop matters.

- **Direct Register and Memory Control:** The programmer decides exactly which values stay in registers during the sample loop, avoiding unnecessary memory reads and writes.
- **No Runtime Overhead:** The engine has no garbage collector, interpreter, or runtime library. The compiled library stays only a few kilobytes in size.
- **Predictable Timing:** With no garbage collection pauses or hidden background work, generation time is consistent from run to run.
- **Direct Access to SIMD Instructions:** SSE instructions can be used directly instead of relying on the compiler to apply them automatically.
- **Educational Value:** The project connects CA010 topics such as registers, addressing modes, and calling conventions to output that can be heard and seen. A change at the register level produces an audible change in the sound.
- **Measurable Comparison:** Writing the engine in Assembly allows a direct performance comparison with an equivalent C implementation of the same algorithm, as stated in the Performance Profiling objective. The C version also serves as a reference for checking that the Assembly output is correct.

The project also recognizes the trade-offs. Modern compilers such as GCC and Clang produce highly optimized code, and Assembly is harder to write, debug, and port. For this reason, Assembly is limited to the performance-critical engine, while the GUI is written in a high-level language. This is the same split used by real-world projects such as FFmpeg.

2.7 Program Architecture

The system is split into three layers: a high-level GUI, a small C-compatible interface, and the Assembly engine that performs all sound synthesis. The GUI never computes audio itself; it only sends parameters and receives finished samples.

2.7.1 System Architecture and Flow

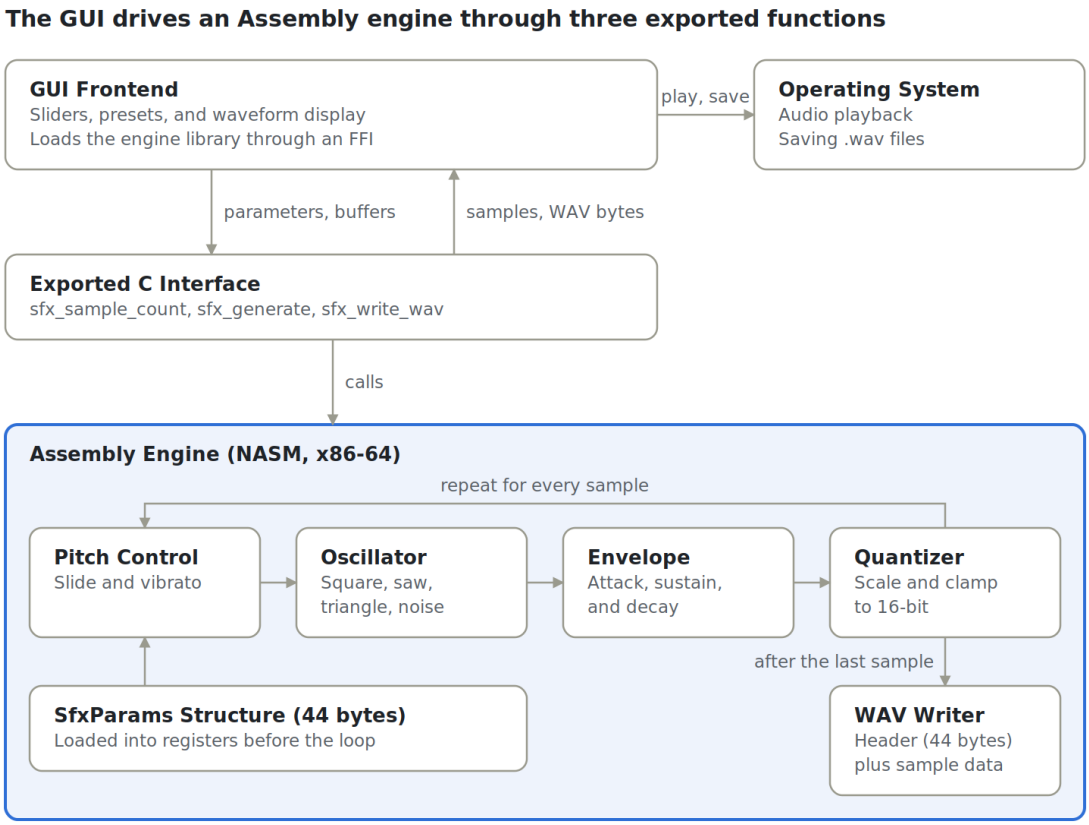


Figure 2.1. System architecture: the GUI drives an Assembly engine through three exported functions.

For each sample, the engine adjusts the pitch, produces one waveform value, applies the volume envelope, and converts the result to a 16-bit integer. After the last sample, the WAV writer adds the header so the GUI can play or save the sound directly.

2.7.2 Pseudocode

The main generation routine follows the flow above. A, S, and D are the attack, sustain, and decay times converted to samples. The pseudocode uses single-precision floating point throughout, matching the SSE instructions the engine uses.

```
function sfx_generate(params, out, capacity):
    if params are invalid: return 0          // see note 3
    A = trunc(attack * 44100)                // seconds to samples (CVTTSS2SI)
    S = trunc(sustain * 44100)
    D = trunc(decay * 44100)
    total = A + S + D
    if total = 0 or total > 220500 or total > capacity: return 0
    phase = 0.0
    vib_phase = 0.0
    freq = base_freq
    y = (slide / 44100) * ln(2)
    slide_mult = 1 + y + y * y / 2           // 2 ^ (slide / 44100), see note 1
    state = seed
    noise_value = to_float(xorshift(state))  // see note 4

    for n = 0 to total - 1:
        // Pitch Control
        freq = min(freq * slide_mult, 20000) // see note 2
        if n > 0 and freq < min_freq: return n  // ends early, note 3
        vib = 1 + vibrato_depth * sine_table[trunc(vib_phase * 256) AND 255]
        vib_phase = vib_phase + vibrato_rate / 44100
        if vib_phase >= 1.0: vib_phase = vib_phase - 1.0
        phase = phase + (freq * vib / 44100)
        if phase >= 1.0:
            phase = phase - 1.0
            noise_value = to_float(xorshift(state))  // new value per cycle

        // Oscillator
        if waveform = SQUARE:   s = (phase < 0.5) ? 1.0 : -1.0
        if waveform = SAWTOOTH: s = 2 * phase - 1
        if waveform = TRIANGLE: s = 1 - 4 * |phase - 0.5|
        if waveform = NOISE:    s = noise_value

        // Envelope
        if n < A:          gain = n / A
        else if n < A + S: gain = 1.0
        else:              gain = 1 - (n - A - S) / D
```

```
        // Quantizer
        out[n] = clamp(round(s * gain * volume * 32767), -32768, 32767)

    return total

function sfx_write_wav(samples, count, out, capacity):
    size = 44 + count * 2
    if count = 0 or count > 220500 or size > capacity: return 0
    write "RIFF", size - 8, "WAVE"
    write "fmt ", 16, 1 (PCM), 1 (mono), 44100, 88200 (byte rate),
          2 (block align), 16 (bits)
    write "data", count * 2
    copy samples after the header
    return size
```

Notes on the pseudocode:

1. Pitch slide without an exponent instruction: The per-sample multiplier is $2^{\text{slide} / 44100}$. Since |slide| is at most 4, the exponent is at most about 9.1×10^{-5}. For such small values, $2^x \approx 1 + y + y^2/2$ with $y = x \cdot \ln 2$, and the next term is far below single-precision resolution. Two MULSS instructions and one ADDSS replace the exponent function.

2. Frequency cap: An upward slide has no natural end, so the running frequency is capped at 20,000 Hz with MINSS. Even with full vibrato, the phase then advances by at most 30,000 / 44,100 ≈ 0.68 per sample, so subtracting 1.0 once is always enough to wrap it.

3. Return values: Validation requires the minimum frequency to be no higher than the base frequency, and the minimum-frequency check starts at the second sample, so every valid call writes at least one sample. A return value of 0 therefore always means an error.

4. Vibrato table and noise: The vibrato sine is read from a 256-entry single-precision table in `.rodata`, indexed from its own phase accumulator. The noise generator is a 32-bit xorshift (shifts of 13, 17, and 5); its state is treated as a signed integer, converted with CVTSI2SS, and scaled by 2^{-31} to give a value from -1.0 up to (but not including) 1.0. A seed of 0 is rejected because xorshift would then output 0 forever.

2.8 Functional Specifications

The specifications below define what the engine and the GUI must do. Numeric targets will be validated during testing and may be adjusted based on results.

2.8.1 Functional Requirements

ID	Requirement	Layer
FR-01	Generate square, sawtooth, triangle, and white noise waveforms	Engine
FR-02	Output mono, 16-bit signed PCM audio at 44,100 Hz	Engine
FR-03	Apply a pitch slide measured in octaves per second, ending the sound early if the frequency falls below a minimum	Engine
FR-04	Apply vibrato with adjustable depth and rate	Engine
FR-05	Apply an attack, sustain, and decay volume envelope	Engine
FR-06	Build a standard 44-byte RIFF/WAVE header followed by the sample data in a memory buffer	Engine
FR-07	Write only to buffers supplied by the caller, with no dynamic memory allocation inside the engine	Engine
FR-08	Produce identical output for identical parameters and noise seed	Engine
FR-09	Provide a slider or selector for every sound parameter and regenerate the sound when a value changes	GUI
FR-10	Provide presets for Coin, Laser, Explosion, Jump, Hit, Power-Up, and Blip, plus a Randomize option	GUI
FR-11	Display the generated waveform	GUI
FR-12	Play the sound through the operating system's audio output	GUI
FR-13	Save the sound as a .wav file	GUI
FR-14	Measure generation time and compare it with an equivalent C implementation of the same algorithm	Both
FR-15	Reject invalid parameters (out of range, NaN, a zero noise seed, or a minimum frequency above the base frequency) by returning 0	Engine

2.8.2 Sound Parameters

The parameters are passed to the engine as a single structure of eleven 4-byte fields (44 bytes in total). Fixed offsets let the Assembly code read each field directly, for example `[rcx + 4]` for the base frequency.

Offset (bytes)	Parameter	Type	Range	Default
0	Waveform (0 = square, 1 = sawtooth, 2 = triangle, 3 = noise)	32-bit integer	0 to 3	0
4	Base frequency (Hz)	32-bit float	20 to 4,000	440
8	Pitch slide (octaves per second)	32-bit float	-4.0 to 4.0	0.0
12	Minimum frequency (Hz)	32-bit float	20 to 1,000, and not above the base frequency	20
16	Vibrato depth (fraction of frequency)	32-bit float	0.0 to 0.5	0.0
20	Vibrato rate (Hz)	32-bit float	0 to 30	6
24	Attack time (s)	32-bit float	0.0 to 2.0	0.01
28	Sustain time (s)	32-bit float	0.0 to 2.0	0.1
32	Decay time (s)	32-bit float	0.0 to 3.0	0.3
36	Master volume	32-bit float	0.0 to 1.0	0.5
40	Noise seed	32-bit unsigned integer	Any nonzero value (0 is rejected)	1

The combined attack, sustain, and decay time is capped at 5 seconds. This limits a sound to 220,500 samples, or 441,044 bytes including the WAV header. The engine checks every field against these ranges, including NaN values, and returns 0 if any check fails. The GUI applies the same limits to its controls, so this should only happen through a programming error.

2.8.3 Exported Library Functions

The engine exposes three functions with a C-compatible interface, so any GUI language with a foreign function interface (for example, Python ctypes) can call it.

```
// Returns the planned sample count, used to size the output buffer,
// or 0 if the parameters are invalid
uint32_t sfx_sample_count(const SfxParams *params);

// Fills 'out' (capacity in samples); returns samples written (fewer
// than planned if the sound ends early), or 0 on error
uint32_t sfx_generate(const SfxParams *params, int16_t *out,
                      uint32_t capacity);

// Wraps samples in a WAV header (capacity in bytes); returns bytes
// written, or 0 if count is invalid or the buffer is too small
uint32_t sfx_write_wav(const int16_t *samples, uint32_t count,
                       uint8_t *out, uint32_t capacity);
```

Callers use the value returned by `sfx_generate`, not `sfx_sample_count`, as the sample count for `sfx_write_wav` and for drawing the waveform, because the sound can end early. All three functions are reentrant: they keep no global state, so they can be called from any thread with separate buffers.

2.8.4 Non-Functional Targets

- **Speed:** Generate a one-second sound (44,100 samples) in under 5 milliseconds on a typical student laptop.
- **Size:** Keep the compiled engine library under 20 KB.
- **Memory:** Use no heap memory and no writable global data inside the engine; all buffers come from the caller.
- **Responsiveness:** Regenerate and redraw the sound within 100 milliseconds of a slider change.
- **Correctness:** Exported WAV files must open without errors in common players and audio editors such as Audacity.

Open question: the group still needs to confirm the GUI language and the primary operating system. This chapter assumes Windows x64 as the main target, with the engine built as a DLL.

2.9 Technical References

Advanced Micro Devices. (n.d.). *AMD64 architecture programmer's manual*. AMD.

Collins, K. (2008). *Game sound: An introduction to the history, theory, and practice of video game music and sound design*. MIT Press.

Duntemann, J. (2023). *x64 assembly language step-by-step: Programming with Linux* (4th ed.). Wiley.

Farnell, A. (2010). *Designing sound*. MIT Press.

Fog, A. (n.d.). *Software optimization resources*. https://www.agner.org/optimize/

IBM Corporation & Microsoft Corporation. (1991). *Multimedia programming interface and data specifications 1.0*.

Intel Corporation. (n.d.). *Intel 64 and IA-32 architectures software developer's manual*. Intel.

Irvine, K. R. (2019). *Assembly language for x86 processors* (8th ed.). Pearson.

Lavelle, S. (n.d.). *bfxr* [Computer software]. https://www.bfxr.net/

Matz, M., Hubička, J., Jaeger, A., & Mitchell, M. (Eds.). (n.d.). *System V application binary interface: AMD64 architecture processor supplement*.

Microsoft. (n.d.-a). *Microsoft Macro Assembler reference*. Microsoft Learn. https://learn.microsoft.com/en-us/cpp/assembler/masm/microsoft-macro-assembler-reference

Microsoft. (n.d.-b). *x64 calling convention*. Microsoft Learn. https://learn.microsoft.com/en-us/cpp/build/x64-calling-convention

NASM Development Team. (n.d.). *NASM: The Netwide Assembler documentation*. https://www.nasm.us/doc/

Pettersson, T. (2007). *sfxr* [Computer software]. https://www.drpetter.se/project_sfxr.html

Smith, S. W. (1997). *The scientist and engineer's guide to digital signal processing*. California Technical Publishing.

Chapter 4 Results and Discussion

The generator matched the C reference exactly across the automated comparison workload. All four Assembly waveforms met the generation-time target on the test laptop, although optimized C remained faster. This chapter presents the implementation results and the limits of the evaluation.

4.1 Implemented System and Evaluation Conditions

The program combines a hand-written x86-64 NASM engine with a Python and Tkinter interface. The engine exports sample counting, sample generation, and WAV construction through sfx.dll. A ctypes binding allocates the buffers and passes the parameter structure to the engine. The frontend provides four waveform choices, nine parameter sliders, a noise-seed field, seven presets, Randomize, a waveform view with a 20 ms zoom option, Play, Save WAV, and an auto-play toggle that starts disabled.

Windows x64, Tkinter, threaded winsound playback, and MinGW-w64 were selected for this implementation. A single build script produces the engine, C reference libraries, native test harness, and benchmark executable. Testing was performed on one laptop; the results therefore do not establish performance across the whole group's machines.

Table 4.1 Evaluation environment

Item	Recorded configuration
Processor	AMD Ryzen 9 8940HX with Radeon Graphics
CPU resources	16 cores and 32 logical processors
Operating system	Windows 11 x64 build 26200
Assembly and C tools	NASM 2.16.03 and GCC 15.2.0
Python	3.12.14 for binding tests; 3.11.9 for Tkinter tests
Timing power scheme	Balanced
Timing conditions	Background apps and AC connection not controlled

Measurements were recorded on 26 October 2026. The C reference used independently written synthesis routines, identical binary32 sine constants, and disabled floating-point

contraction to preserve rounding. Agreement with this reference verifies the specified algorithm but does not establish perceptual quality or exclude a shared interpretation error.

4.2 Correctness and Interface Results

The native harness completed 1,279,144 assertions with no failures. It compared 20 fixed parameter sets, five for each waveform and including all seven presets, plus 10,000 reproducibly seeded random valid sets. Every comparison returned the same sample count and identical signed 16-bit samples. The maximum observed sample difference was 0 LSB, meeting the allowed difference of one LSB.

Table 4.2 Automated verification results

Check group	Workload or condition	Observed result
Reference comparison	20 fixed and 10,000 random sets	Identical samples
Repeatability	1,000 repeated native calls	Identical bytes
Input validation	Range, NaN, infinity, null and cross-field errors	Rejected without writes
Buffer protection	Short, exact and oversized capacities	Guard bytes intact
ABI preservation	Success and error paths	Nonvolatile registers preserved
WAV construction	1, 44,100 and 220,500 samples	Header and payload matched
Randomize	1,000 binding-level runs	Valid sounds generated
Concurrent generation	100 calls using four worker threads	Identical expected output

Native checks also covered pitch-slide timing, early termination, waveform periods, full-scale quantization, and maximum upward slide. A descending sound whose minimum frequency equals its starting frequency generated one sample. WAV construction uses the actual generated count, preventing unused sample-buffer contents from appearing after early termination.

PE inspection found exactly three exports, no imported DLLs, and no writable sections. The read-only import directory contains only its null terminator; Windows unwind metadata also remains read-only. The 44-byte ctypes parameter layout and every field offset matched the C and Assembly interface.

4.3 Generated Sound Files

All seven preset WAV files were read back with Python's wave module. Each reported mono audio at 44,100 Hz with a two-byte sample width, and its payload matched the generated samples exactly. Table 4.3 reports the outputs for seed 1.

Table 4.3 Exported preset outputs

Preset	Samples	Duration in s	WAV bytes
Coin	13,230	0.30	26,504
Laser	8,820	0.20	17,684
Explosion	39,690	0.90	79,424
Jump	11,025	0.25	22,094
Hit	7,497	0.17	15,038
Power-Up	22,050	0.50	44,144
Blip	3,528	0.08	7,100

Repeated generation reproduced each preset file exactly, while changing the noise seed changed the noise output. These results establish structural WAV correctness and repeatability. Preset recognisability requires the group listening test; Audacity and Windows Media Player compatibility also remain unverified.

4.4 Generation Performance and Resource Use

The native benchmark measured synthesis alone using QueryPerformanceCounter. Each workload generated one second of audio, or 44,100 samples, with pitch slide and vibrato enabled. For each waveform and implementation, 100 warm-up calls preceded 1,000 measured calls. The output buffer was allocated before timing. DLL loading, WAV construction, Python conversion, canvas drawing, playback, and file writing were excluded. Table 4.4 reports medians for all implementations and the Assembly 95th percentile.

Table 4.4 Time to generate 44,100 samples in milliseconds

Waveform	Assembly median	Assembly p95	C O2 median	C O0 median
Square	0.352	0.393	0.308	0.936
Sawtooth	1.577	1.843	0.316	1.711
Triangle	1.678	1.930	0.398	2.053
Noise	0.329	0.508	0.262	0.960

Assembly medians ranged from 0.329 to 1.678 ms, below the 5 ms target for every waveform on this machine. However, Assembly took 1.14 to 4.98 times the optimized C median. It was faster than unoptimized C in all four measured workloads. The measurements therefore support the speed requirement on this laptop but do not support a claim that hand-written Assembly is inherently faster than optimized C.

Branch structure, scalar dependencies, and instruction choices may explain the timing differences, but this remains a hypothesis without hardware-counter analysis or a controlled optimization experiment. Fixed execution order and uncontrolled background activity also limit interpretation of small differences.

The stripped Assembly DLL occupied 5,632 bytes, below the 20 KB target. The optimized C reference object occupied 3,870 bytes; an object and a complete DLL are different packaging units, so these sizes do not establish a language-level size advantage. Source inspection found a 152-byte generation stack frame, no heap allocation, and no writes to global state. Including nested return addresses, engine stack use remains below 256 bytes. The NASM listing contains 78 statically emitted instructions between the loop and completion labels across alternative paths; this is not the dynamic instruction count for each sample.

4.5 Frontend Integration Results

The binding suite passed 1,052 assertions. A separate off-screen Tkinter suite passed 140 assertions using real widgets and a 480-column Canvas. It exercised every preset, slider limits, seed validation, zoom, saving the current WAV, and the missing-DLL error

state. Playback was mocked to verify that the callback runs in a worker thread and the UI event loop continues processing events. This test did not send sound to the speakers.

For 100 maximum-length sounds of 220,500 samples, the off-screen test measured a median event-to-canvas time of 80.8 ms and a 95th percentile of 89.2 ms, including the 50 ms debounce. The measured p95 was below the 100 ms target in this setup. The withdrawn window and fixed Canvas dimensions exclude visible-window composition and display costs; a visible interactive measurement is still required before treating NFR-04 as fully verified.

4.6 Remaining Evaluation and Interpretation

Several acceptance activities remain open. A second group member must review the visible interface, operate Play and Save, and confirm speaker output. The group must listen to all presets and judge whether each effect is recognisable. WAV files still need to be opened in Audacity and Windows Media Player. The ten-minute interactive soak session, clean-install check, and benchmark repetition on a second laptop have not been completed. The 1,000 automated Randomize calls provide stability evidence for generation, but do not replace those activities.

Final benchmarking should use connected power, a recorded power mode, closed background applications, and a second laptop. A later battery snapshot cannot establish power conditions throughout timing. Raw test output, environment metadata, disassembly, and source and DLL hashes are retained in the results directory for reproduction.

The system demonstrates reproducible Assembly synthesis, frontend integration, and PCM WAV export within the measured resource targets on one machine. Independent user review, listening tests, external-player checks, longer stability testing, and performance replication are required to complete the evaluation.